ADR-0004 : Image de production statique (musl) sur base distroless

- **Statut :** Accepté
- **Date :** 2026-07-21
- **Décideurs :** Driss OUAAZIZ — Software Architect

Contexte et problème

L'image conteneur de production était bâtie sur `gcr.io/distroless/cc-debian12`. Bien que minimaliste, cette base embarque des bibliothèques système (glibc, OpenSSL, runtime GCC) porteuses de vulnérabilités. Un scan de l'image construite a mis en évidence **17 CVE** (0 critique/haute, 4 moyennes, 13 basses), **toutes sans correctif disponible** (en attente d'un correctif Debian amont) et sans lien avec le code applicatif.

FileFlow ayant adopté un backend cryptographique pur Rust (ADR-0003), le binaire **n'a plus besoin d'OpenSSL** ni, à terme, de la glibc : la voie d'un binaire **entièrement statique** devient possible et permettrait de supprimer ces bibliothèques système de l'image.

Facteurs de décision

- Réduire au maximum la surface de vulnérabilités système de l'image livrée.
- Diminuer la taille de l'image (distribution, temps de tirage).
- Conserver une image sans shell ni utilitaires (principe distroless, exécution en utilisateur non privilégié).
- Aboutir à un binaire qui démarre effectivement sur la base retenue.

Options envisagées

1. **Conserver `distroless/cc-debian12`** — Statu quo : 17 CVE système irrémédiables tant que Debian n'a pas publié de correctifs.
2. **Épingler/rafraîchir régulièrement la base** — Atténue sans supprimer ; les CVE concernées n'ont de toute façon pas de correctif.
3. **Binaire statique musl sur `distroless/static`** *(retenu)* — Élimine glibc/OpenSSL/GCC de l'image ; ne subsiste que le binaire applicatif.

Décision

Compiler pour la cible **`x86_64-unknown-linux-musl`** un **binaire entièrement statique** et le livrer sur **`gcr.io/distroless/static-debian12:nonroot`**.

Points de mise en œuvre :

- Installation de `musl-tools` / `musl-dev` dans l'étage de build : le crate `rusqlite` compile SQLite (code C embarqué) et nécessite `musl-gcc`.
- Édition de liens statique **non-PIE** via `RUSTFLAGS="-C target-feature=+crt-static -C relocation-model=static"`. Sans cela, musl produit par défaut un exécutable **static-PIE** qui référence toujours l'éditeur de liens dynamique `/lib/ld-musl-x86_64.so.1`, absent de `distroless/static` : le conteneur échoue alors à l'`exec` (« no such file or directory »).

- Exécution en utilisateur `nonroot` ; sonde de santé autonome (`fileflow healthcheck`), l'image ne disposant ni de shell ni de `curl` .

Conséquences

Positives

- **0 CVE d'image** au scan (contre 17 auparavant) : glibc, OpenSSL et runtime GCC ont disparu de l'image.
- Taille réduite de **34,1 Mo à 11,3 Mo** (environ −67 %).
- Surface d'attaque minimale : ni shell, ni gestionnaire de paquets, ni bibliothèques système à maintenir.

Négatives

- La compilation en **non-PIE** prive le binaire de l'**ASLR** (randomisation de l'espace d'adressage) sur son propre segment. Cet arbitrage est **assumé** : pour un binaire Rust (langage à sûreté mémoire, `unsafe` marginal), le risque pratique est faible au regard du gain (image sans CVE). Alternative conservant l'ASLR : embarquer le chargeur musl dans l'image pour rester en static-PIE — écartée pour préserver la pureté de `distroless/static` .
- Chaîne de build plus spécifique (cible musl, toolchain C musl).

Risques résiduels

- Toute future dépendance imposant une bibliothèque C dynamique remettrait en cause la compilation statique et donc le choix de base d'image.

Références

- Images distroless : https://github.com/GoogleContainerTools/distroless.
- Code : `Dockerfile` , `backend/Cargo.toml` .
- ADR-0003 (crypto pur Rust, préalable à l'abandon d'OpenSSL).